Documento Técnico TourFer - Avance 2

Presentado a:
Leidy Mariani Dorado

Presentado por
Manuel Felipe Fernández
Jose Luis Montenegro
Jose Hebert Aponza

Materia
Sistemas Distribuidos

Programa
Ingeniería de sistemas

Corporación Universitaria Comfacauca
Popayán – Cauca
2026 - 1

1. Descripción del Avance..... 3
Funcionalidades Operativas:..... 3
Componentes Pendientes:..... 3
2. Arquitectura Actualizada..... 4
Servicios Existentes:..... 4
3. Comunicación entre Servicios..... 5
Los endpoints que tiene TourFer son:..... 5
Flujo Completo de Interacción:..... 5
4. Configuración del Entorno..... 6
Gestión de Configuraciones:..... 6
Portabilidad:..... 6

1. Descripción del Avance

El proyecto ha evolucionado de una propuesta conceptual a un ecosistema funcional de microservicios. En este segundo corte, el enfoque principal fue la **descomposición de servicios y la persistencia de datos**.

Funcionalidades Operativas:

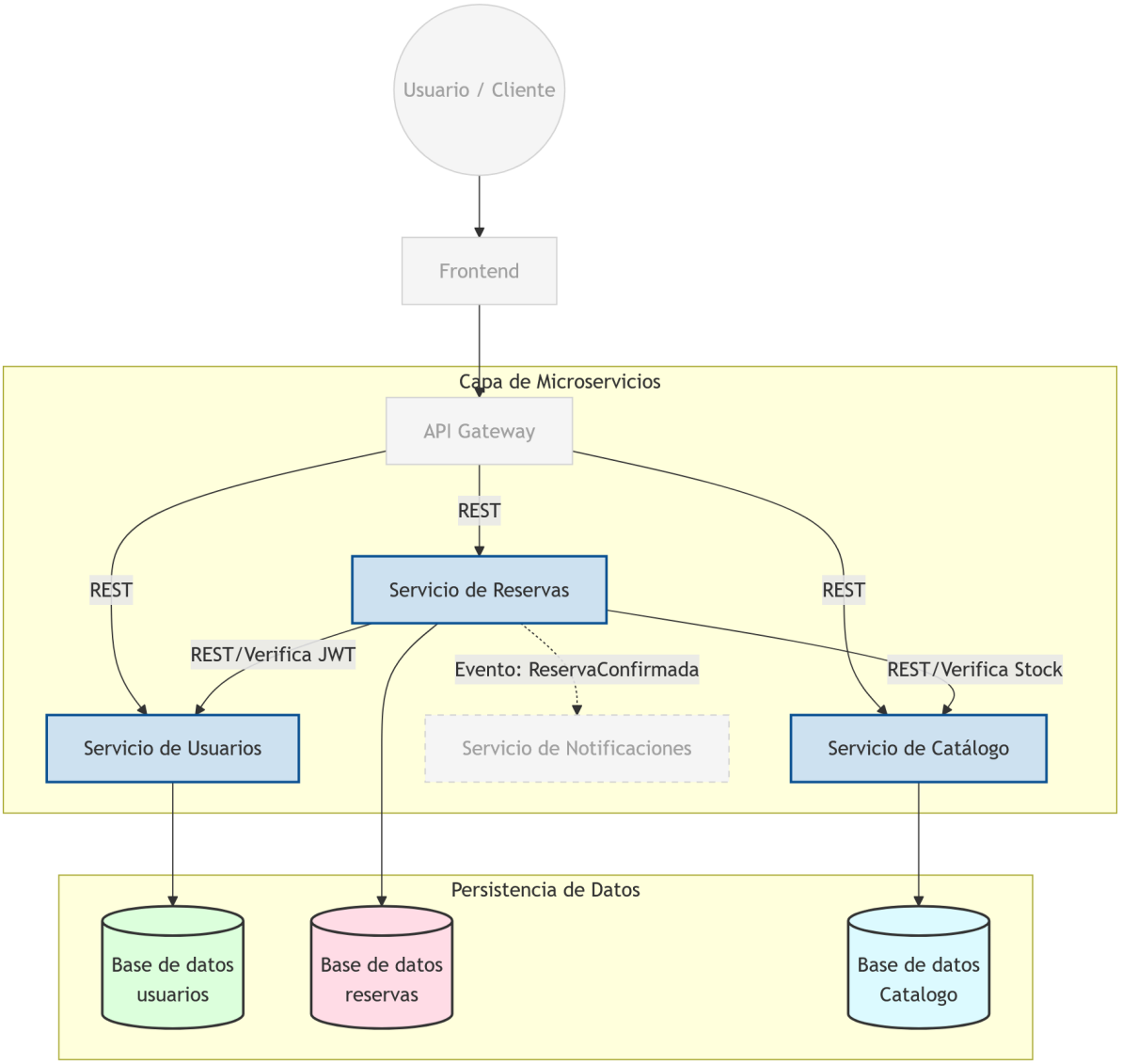
- **Servicio de Usuarios:** Implementación completa de registro y autenticación mediante **JWT (JSON Web Tokens)**.
- **Servicio de Catálogo:** Consulta de tours y guías con persistencia en base de datos.
- **Servicio de Reservas:** Capacidad de crear transacciones que impactan el inventario de otros servicios.
- **Persistencia de datos:** Cada servicio cuenta con su propio esquema de base de datos (usuarios_db, catalogo_db, reservas_db).

Componentes Pendientes:

- **API Gateway:** Actualmente la comunicación es directa entre servicios; se planea centralizar el tráfico en el siguiente avance.
- **Servicio de Notificaciones:** La lógica de envío de correos o alertas está simulada mediante logs, pendiente de integración con un proveedor externo o broker de mensajería.
- **Frontend:** El sistema es actualmente solo backend, consumido mediante herramientas de testeo como Postman.

2. Arquitectura Actualizada

Este es el diagrama actual de TourFer, donde se representa con color aquellos servicios que actualmente se encuentran en funcionamiento, mientras que aquellos que no, estan de color gris.



Servicios Existentes:

1. **tourfer-users:** Registro, login y autenticación.
2. **tourfer-catalogo:** Fuente de verdad para tours y disponibilidad.
3. **tourfer-reservas:** Orquestador de la lógica transaccional de compra.
4. **tourfer-db:** Instancia centralizada de MySQL que aloja los 3 esquemas independientes.

3. Comunicación entre Servicios

La comunicación se basa en el protocolo **HTTP/REST**. Se ha implementado un flujo de integración real para asegurar la consistencia de los datos.

Los endpoints que tiene TourFer son:

- POST /register desde el servicio de usuarios para el respectivo registro
- POST /login desde el servicio de usuarios igualmente para el login y obtención del token de autenticación
- POST /reserva desde el servicio de reservas donde se debe agregar el id del tour y la cantidad de personas para hacer la reserva
- GET /mis-reservas desde el servicio de reservas para visualizar todas las reservas que tiene un mismo usuario
- GET /Tours desde el servicio de catálogo para visualizar todos los tours disponibles
- GET /Guias desde el servicio de catálogo, lo mismo que el anterior, pero para visualizar todo los guías

Hay un endpoint más, que permite la integración de los servicios:

PATCH /tours/<id>/cupos expuesto por el API Catálogo para que otros servicios (reservas) modifiquen el inventario de forma segura, que está protegido por X-API-Key.

Flujo Completo de Interacción:

1. El **Servicio de Reservas** recibe una petición POST.
2. Valida la existencia y el stock del tour realizando un GET al **Servicio de Catálogo**.
3. Si la validación es positiva, el Servicio de Reservas solicita un PATCH al Catálogo para decrementar los cupos.
4. Finalmente, se persiste la reserva en la base de datos de Reservas.

Cabe destacar que este endpoint no se encuentra expuesto ni en el readme, ni explícitamente en las capturas de evidencia, debido a que es un funcionamiento interno que implementó en el servicio para que al hacer una reserva dichos cupos se actualicen automáticamente del tour, por lo que no se pueden hacer pruebas directas, a menos de que se a través del endpoint de reservas.

Ahora bien, se optó por REST debido a su facilidad de depuración y trazabilidad, y para manejar la consistencia, se implementó una lógica de compensación si la reserva falla tras haber descontado los cupos, el sistema envía una señal automática de incremento para revertir el cambio en el Catálogo, evitando datos huérfanos.

4. Configuración del Entorno

La portabilidad y seguridad del sistema se gestionan estrictamente mediante la inyección de dependencias por variables de entorno.

Gestión de Configuraciones:

- **Variables de Entorno (.env):** Se centralizan todas las credenciales como passwords de DB, JWT secrets, X-API-Key, y configuraciones de red como puertos y URLs, con el fin de evitar el hardcoding y facilitar el despliegue en diferentes servidores.

- **DNS Interno de Docker:** En lugar de usar direcciones IP volátiles o localhost, los servicios se comunican usando los nombres definidos en el docker-compose.yml, por ejemplo http://tourfer-catalogo:5000.

- **Seguridad:** Se implementó una doble capa de protección, puesto que se maneja JWT para usuarios finales y API Keys para la comunicación directa entre contenedores, asegurando que los servicios no sean manipulados externamente sin autorización.

Portabilidad:

Gracias al uso de docker, el sistema es agnóstico al sistema operativo, al incluir un archivo .env.example, cualquier desarrollador puede replicar el entorno completo ejecutando un único comando, garantizando que el sistema sea funcional desde el primer momento.